



Sunbeam Institute of Information Technology
Pune and Karad

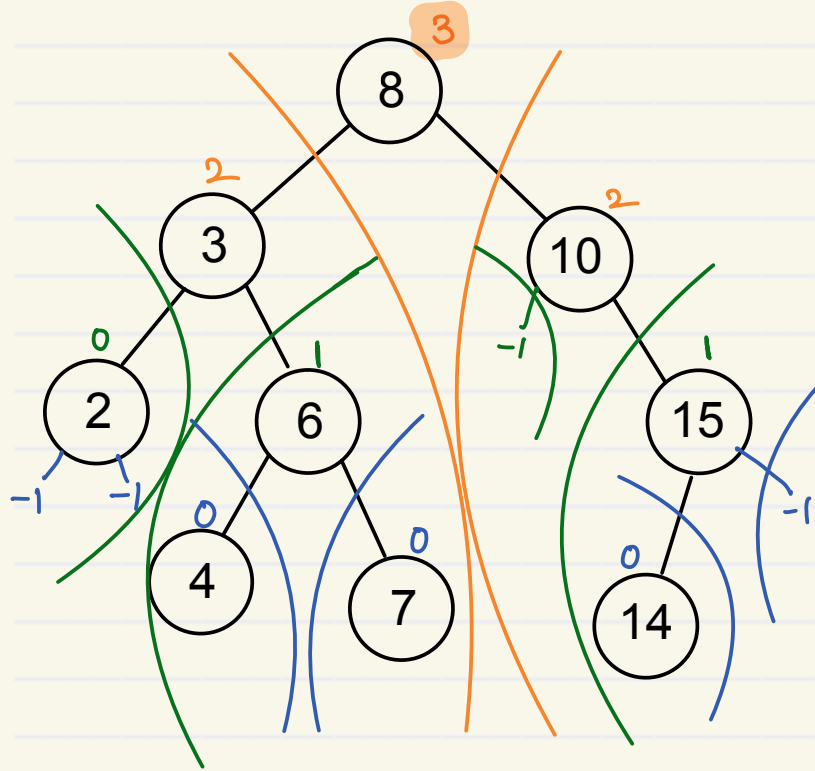
Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Binary Search Tree - Height

Height of root = MAX (height (left sub tree), height (right sub tree)) + 1



1. If left or right sub tree is absent then return -1
2. Find height of left sub tree
3. Find height of right sub tree
4. Find max height
5. Add one to max height and return

height (Empty / Null tree) = -1

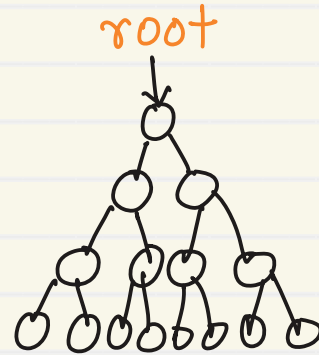
BST - Time complexity of operations

Capacity:

max number of nodes that can be added into a binary tree for give height.

height nodes

-1	0
0	1
1	3
2	7
3	15



h - height of binary tree
 n - no. of elements in binary tree

$$n = 2^{h+1} - 1$$

$$n = 2^{h+1} - 1$$

$$2^{h+1} = n + 1$$

$$2^h \approx n$$

$$\log 2^h = \log n$$

$$h \log 2 = \log n$$

$$h = \frac{\log n}{\log 2}$$

time $\propto$ height
time $\propto \frac{\log n}{\log 2}$

$$T(n) = O(\log n)$$

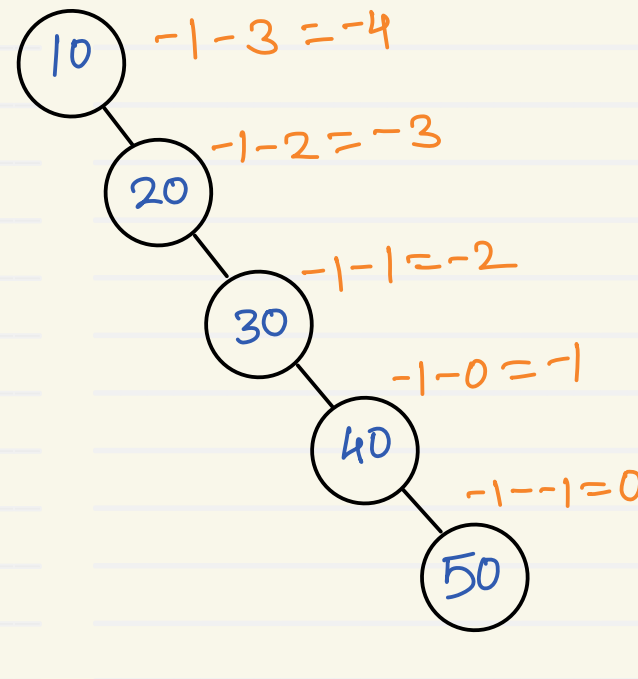
Add
Delete
Search } $O(h)$ or $O(\log n)$

Traverse - $O(n)$

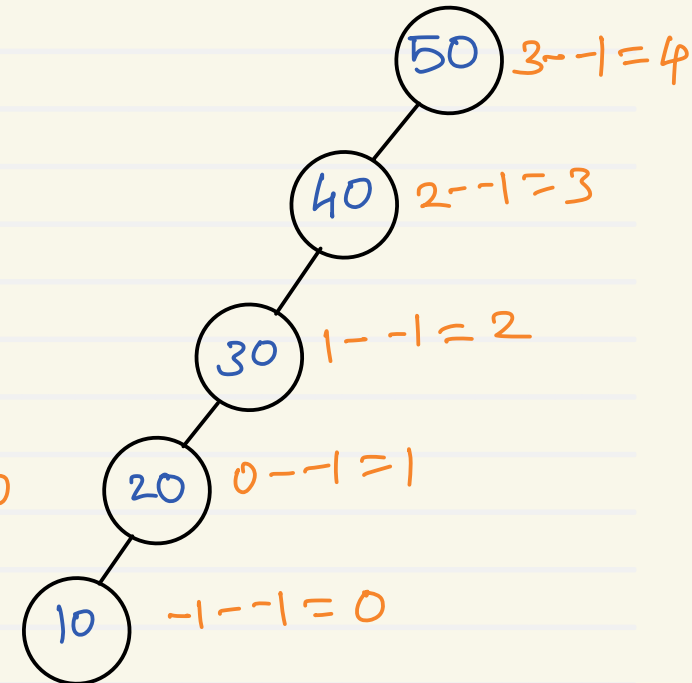
Skewed Binary Tree

- In binary tree if only left or right links are used, tree grows only in one direction such tree is called as skewed binary tree
 - Left skewed binary tree
 - Right skewed binary tree
- Time complexity of any BST is $O(h)$
- Skewed BST have maximum height ie same as number of elements.
- Time complexity of searching in skewed BST is $O(n)$

Keys : 10, 20, 30, 40, 50



Keys : 50, 40, 30, 20, 10



height $\approx n$
 $T(n) = O(n)$

- To speed up searching, height of BST should be minimum as possible
- If nodes in BST are arranged, so that its height is kept as less as possible, is called as Balanced BST

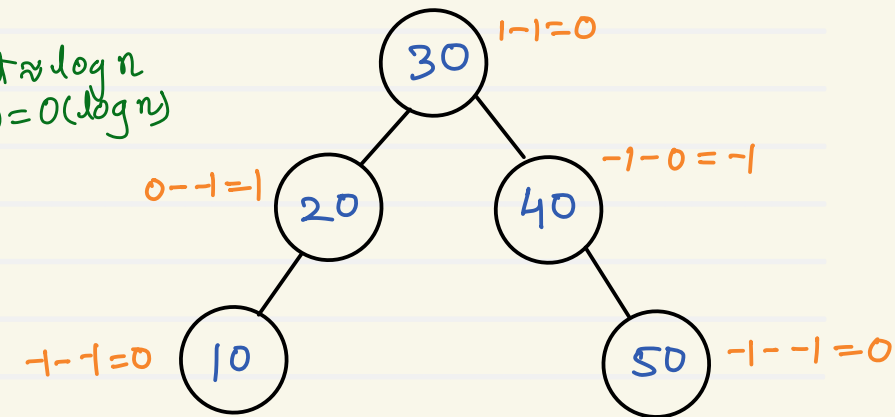
Balance factor (node) = Height (left sub tree) - Height (right sub tree)

- tree is balanced if balance factors of all the nodes is either -1, 0 or +1
- balance factors = {-1, 0, +1}
- A tree can be balanced by applying series of left or right rotations on imbalance nodes ← node having balance factor other than -1, 0 or +1

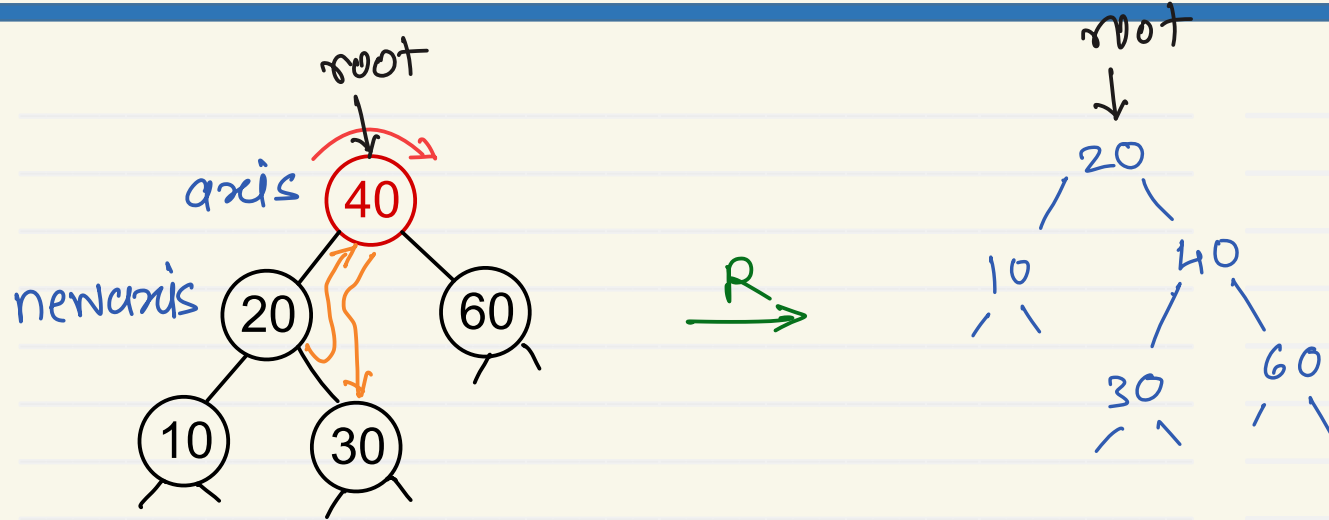
Types :

1. AVL Tree
2. Red Black Tree
3. Splay Tree

Keys : 30, 40, 20, 50, 10

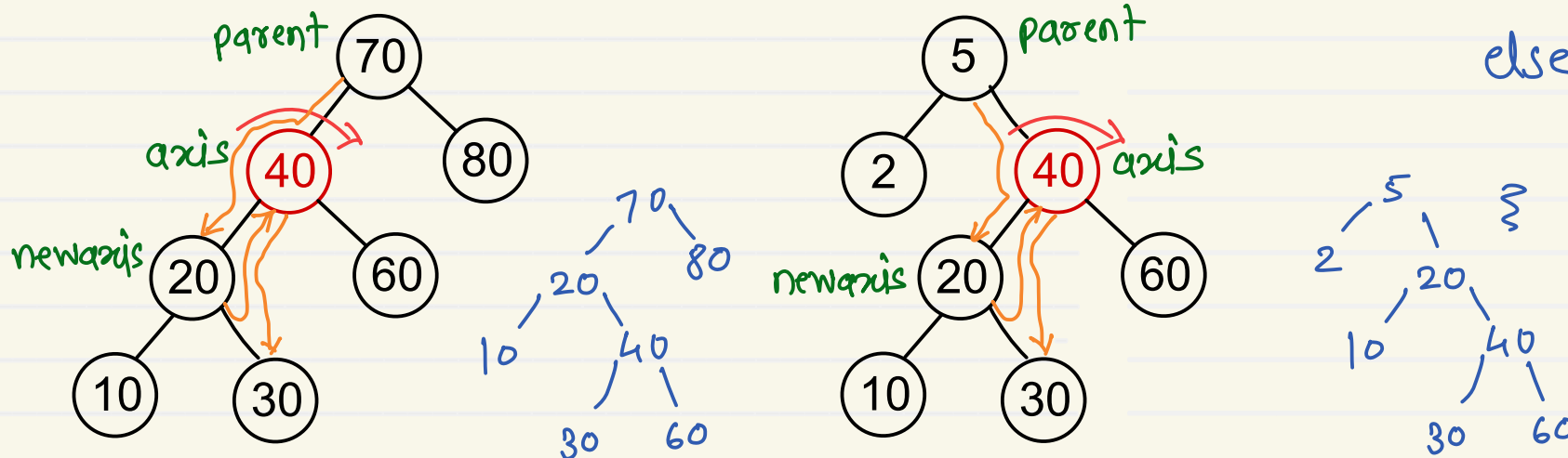


Right Rotation

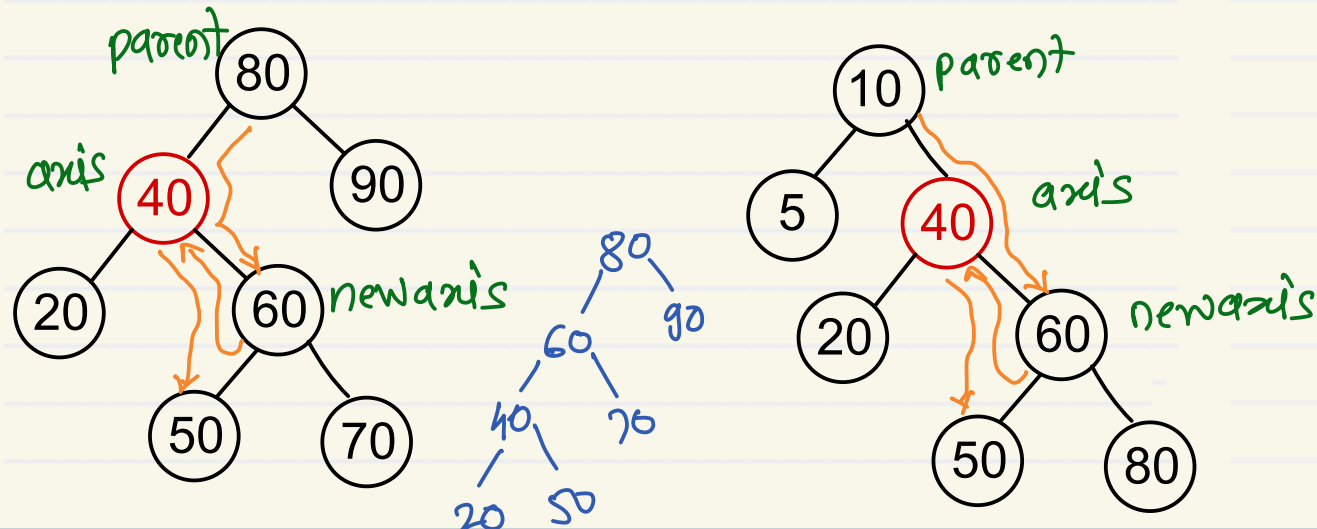
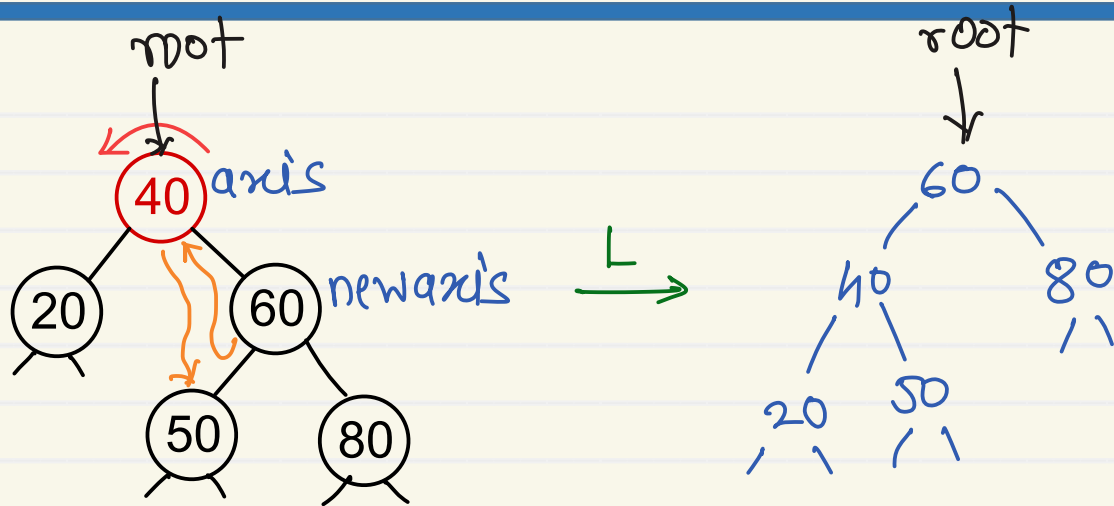


```

rightRotation(axis, parent) {
    newaxis = axis.left;
    axis.left = newaxis.right;
    newaxis.right = axis;
    if (axis == root)
        root = newaxis;
    else if (axis == parent.left)
        parent.left = newaxis;
    else if (axis == parent.right)
        parent.right = newaxis;
}
    
```



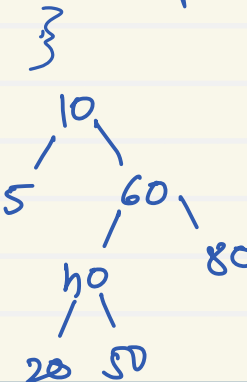
Left Rotation



```

leftRotation(axis, parent) {
    newaxis = axis.right;
    axis.right = newaxis.left;
    newaxis.left = axis;
    if (axis == root)
        root = newaxis;
    else if (axis == parent.left)
        parent.left = newaxis;
    else if (axis == parent.right)
        parent.right = newaxis;
}

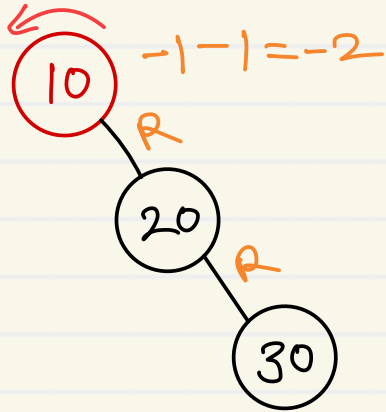
```



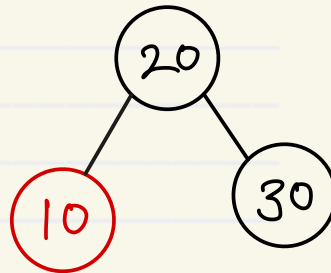
Rotation cases : Right-Right case (single rotation)

→ apply left rotation on imbalance node

Keys : 10, 20, 30



L →

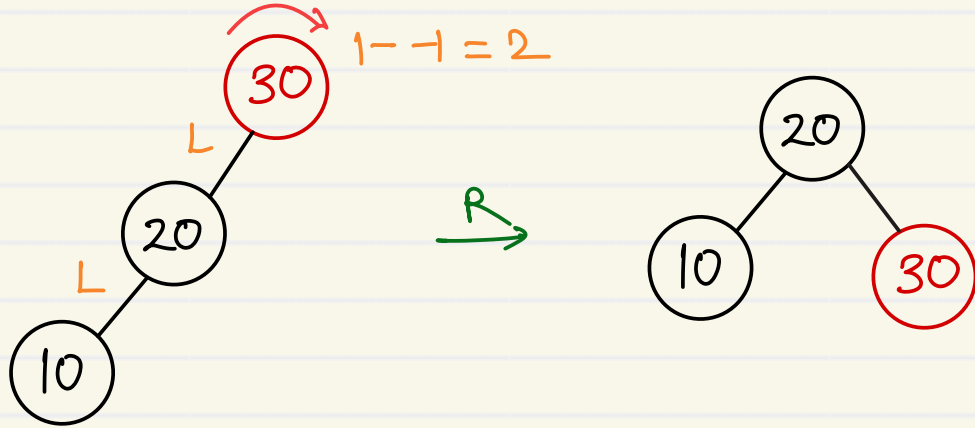


```
if (bf < -1 && value > trav.right.data) {  
    leftRotation(trav, —);  
}
```


Rotation cases : Left-Left case (single rotation)

→ apply right rotation on imbalance node

Keys : 30, 20, 10

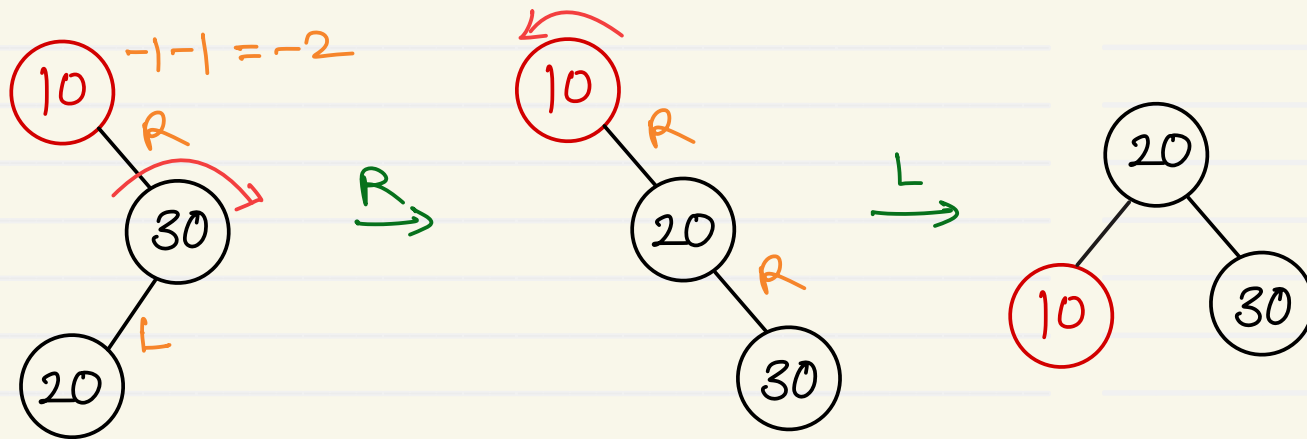


```
if( bf > +1 && value < trav.left->data ) {  
    rightRotation( trav, — )  
}
```

Rotation cases : Right-Left case (Double rotation)

Keys : 10, 30, 20

→ apply right rotation on right of imbalance node
→ apply left rotation on imbalance node

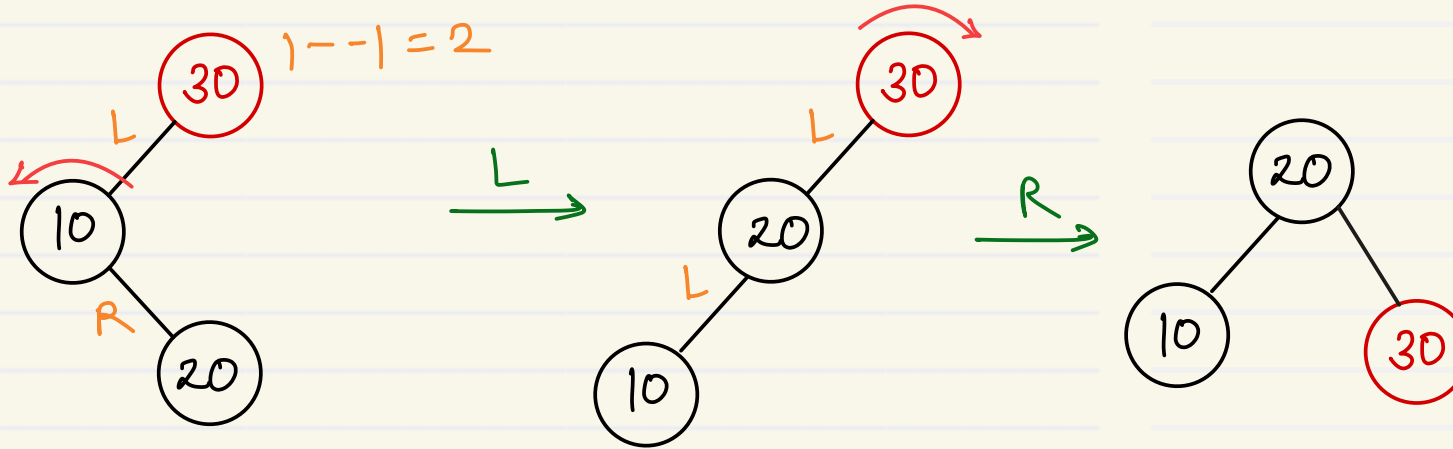


```
if (bf < -1 & value < trav.right.data) {
    rightRotation(trav.right, trav);
    leftRotation(trav, —)
}
```

Rotation cases : Left-Right case (Double rotation)

Keys : 30, 10, 20

→ apply left rotation on left of imbalance node
→ apply right rotation on imbalance node

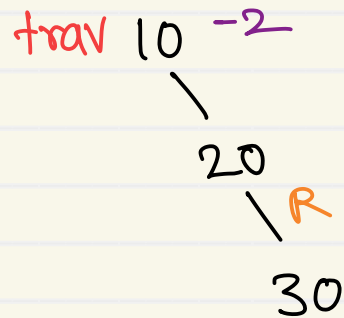


```
if (bf > +1 & value > trav.left->data) {
    leftRotation(trav.left, trav);
    rightRotation(trav, —);
}
```

Balance Factor = $\{-1, 0, +1\}$

BF < -1

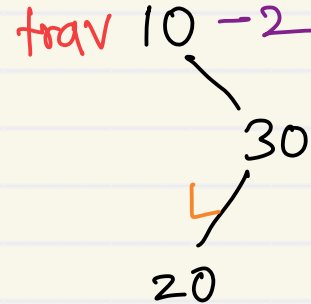
RR



value = 30

value > trav.right.data

RL

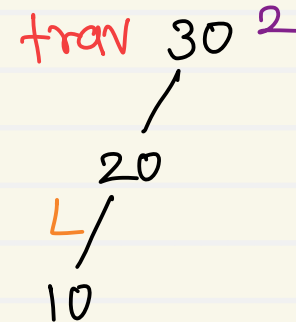


value = 20

value < trav.right.data

BF > +1

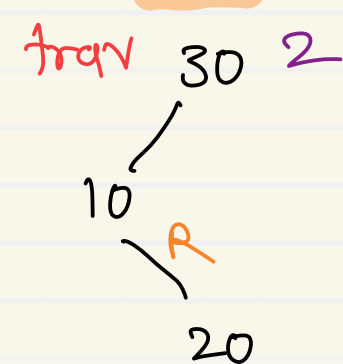
LL



value = 10

value < trav.left.data

LR



value = 20

value > trav.left.data